

Epic 3: Dynaaminen Schema-validointi ja Itsekorjautuva Tekoäly (Self-Healing AI)

Epic ID: QUORUM-EPIC-V2-003

Tila: Valmis kehitettäväksi (Ready for Dev)

Teema: Agentic AI, Reliability, LLM Output Parsing, Dynamic Typing

Kohdemoduulit: backend_v2/llm/schema_builder.py (UUSI), backend_v2/llm/handler.py, backend_v2/hooks/llm.py

Riippuvuudet: Voidaan aloittaa Epic 1:n rinnalla, mutta integraatio Hookeihin vaatii Epic 1:n (Hook-immutabiliteetti) valmistumista.

Laajuusarvio: XL (3–4 viikon iteratiivinen kehitys ja pilotointi)

1. Tausta ja Ongelmakuvaus

Quorum V2 nojaa "Schema-Driven AI" -paradigmaan: tekoälyä ohjataan tuottamaan JSON-muotoista dataa, joka vastaa järjestelmään konfiguroituja arviointimatriiseja ja sääntöjä (PromptBlocks). Tällä hetkellä järjestelmä luottaa sokeasti siihen, että LLM palauttaa oikeanlaisen Python-sanakirjan (dict[str, Any]).

Nykyisen arkkitehtuurin kriittiset ongelmat:

- Tekoälyn rakenteelliset hallusinaatiot:** Vaikka LLM:ää kuinka ohjeistettaisiin, se tekee muotovirheitä (esim. palauttaa luvun "5" merkkijonona kun pyydettiin 5.0, tai unohtaa pakollisen avaimen "justification"). Tämä johtaa siihen, että virheellinen data vuotaa tietokantaan tai kaataa myöhemmät askeleet.
- Defensiivisen koodauksen helvetti:** Jälkiprosessoinnista (esim. scoring.py tai metrics.py) on muodostunut monimutkainen spagettikoodin verkko, jossa yritetään epätoivoisesti arvata, yrittikö LLM palauttaa listan vai merkkijonon.
- Kryptiset ja tyhmit kaatumiset:** Jos LLM-vastaus on täysin rikki, ohjelma kaatuu (Fatal Failure) KeyError- tai TypeError-poikkeuksiin. Käyttäjä saa 500 Server Errorin, eikä järjestelmä osaa hyödyntää LLM:n kontekstiymmärrystä pyytämällä sitä korjaamaan omaa virhettään. Kalliit API-tokenit valuvat viemäriin.
- Muistivuotoriski (Memory Leaks):** Alkuperäinen ajatus luoda Pydantic-malli lennosta (pydantic.create_model) jokaisen LLM-vastauksen kohdalla on vaarallinen. Se luo Pythonin muistiin jatkuvasti uusia luokkia (type), joita roskienkerääjä ei pysty siivoamaan tehokkaasti pitkäkestoisissa Worker-prosesseissa, mikä johtaa OOM (Out of Memory) -kaatumisiin.

2. Tavoitteet ja Liiketoiminta-arvo

Tämän Epicin tavoitteena on rakentaa ohjelmiston ja tekoälyn väliin läpäisemätön, tyyppiturvallinen palomuuuri ja muuttaa Quorum aidoksi **Agentic AI** -alustaksi.

- **100 % Tyyppiturvallisuus (Coercion):** LLM:n tuottama JSON pusketaan lennosta generoidun Pydantic-mallin läpi. Pydantic pakottaa ja korjaa datatyyppit automaattisesti matemaattiseen muotoon.
- **Structured Outputs (First-Try Accuracy):** Generoitu Pydantic-malli käännetään JSON Schemaksi ja syötetään suoraan LLM-palveluntarjoajan API:in (esim. OpenAI `response_format`), mikä ohjaa mallin toimintaa rakenteellisesti jo luontihetkellä.
- **Automaattinen itsensäkorjaus (Self-Healing AI):** Jos Pydantic hylkää vastauksen (esim. pakollinen kenttä puuttuu), järjestelmä ottaa kiinni `ValidationError`in ja lähettää sen sellaisenaan takaisin LLM:lle: *"Vastasit väärin. Korjaa tämä JSON: [Pydanticin koneellinen virhe]"*. Tekoäly korjaa itse itsensä (Auto-Retry) ilman ihmisen väliintuloa.
- **Kehittäjäkokemus (DX):** Hookit saavat jatkossa vain validoituja, staattisesti luotettavia objekteja. Manuaalinen defensiivinen koodaus poistuu.

3. Arkkitehtuurilinjat (Technical Guidelines)

Toteutuksessa on vältettävä kaksi merkittävää arkkitehtuurista sudenkuoppaa:

1. **Muistivuotojen esto (Caching on pakollinen):** Pydanticin `create_model()` luo aina uuden Python-luokan muistiin. Koska luokat eivät poistu roskienkeruussa (GC) tehokkaasti, jokaisen LLM-vastauksen kohdalla tapahtuva luokan generointi tukkii Worker-instanssin muistin (OOM).
 - **Ratkaisu:** Luotava uusi `SchemaCompilerService`. Se laskee konfiguraatiosta tiivisteeseen (Hash) ja hakee generoidun Pydantic-luokan välimuistista (`functools.lru_cache`). Uusi tyyppi instansoidaan vain, jos täsmälleen samanlaista rakennetta ei ole aiemmin nähty.
2. **Circuit Breaker (Katkaisija):** Itsekorjausluuppi (Self-Healing) ei saa jäädä ikuisen silmukkaan polttamaan rahaa. LLM-kutsulle on asetettava tiukka maksimiyritysten määrä (esim. `max_retries = 3`).
3. **Työkalujen (Libraries) hyödyntäminen:** Arvioidaan aluksi, kannattaako pyörää keksiä täysin uudelleen. Modernit kirjastot kuten **instructor** tekevät juuri tätä (Pydantic-integraatio, Retry-luopit ja Structured Outputs). Jos se soveltuu nykyiseen asynkroniseen arkkitehtuuriimme, käytetään sitä. Muussa tapauksessa rakennetaan oma kevyt Handler.

Koodiesimerkki (Tavoitearkkitehtuuri)

1. Dynaamisen luokan rakentaja & Välimuisti (`schema_compiler.py`):

Python

```
import hashlib
import functools
import json
from pydantic import BaseModel, create_model, ConfigDict, Field
from typing import Type, Any

class SchemaCompilerService:
    @staticmethod
    def _generate_hash(schema_config: dict[str, Any]) -> str:
        # Luodaan vakaa tiiviste kenttien konfiguraatiosta
        config_str = json.dumps(schema_config, sort_keys=True)
        return hashlib.sha256(config_str.encode()).hexdigest()

    @staticmethod
    @functools.lru_cache(maxsize=1024)
    def _get_or_create_model(schema_hash: str, fields_tuple: tuple) -> Type[BaseModel]:
        # TÄRKEÄÄ: Luodaan Pydantic-luokka vain kerran per uniikki konfiguraatio (estää muistivuodot)
        fields = {name: (type_hint, Field(...)) for name, type_hint in fields_tuple}
        return create_model(
            f"DynamicSchema_{schema_hash[:8]}",
            __config__=ConfigDict(extra="forbid", strict=False), # strict=False sallii tyyppimuunnokset
            # coercion
            **fields
        )

    @classmethod
    def compile(cls, schema_config: dict[str, Any]) -> Type[BaseModel]:
        schema_hash = cls._generate_hash(schema_config)
        # Oikeassa toteutuksessa konfiguraation tyyppimäärittelyt mapataan turvallisesti Python-tyyppeihin
        fields_tuple = tuple((k, float if v.get("type") == "number" else str) for k, v in
            schema_config.items())
        return cls._get_or_create_model(schema_hash, fields_tuple)
```

2. Self-Healing Luuppi (handler.py / llm.py):

Python

```

from pydantic import ValidationError
import logging
import json

async def generate_with_self_healing(llm_client, prompt: str, schema_config: dict, max_retries=3):
    # 1. Hae välimuistista dynaaminen Pydantic-luokka
    DynamicModel = SchemaCompilerService.compile(schema_config)

    # 2. Pyydä APIa pakottamaan muoto (Structured Outputs)
    json_schema = DynamicModel.model_json_schema()

    current_prompt = prompt
    for attempt in range(max_retries):
        try:
            # LLM-kutsu (injektoidaan JSON Schema natiivina formaattina)
            raw_response = await llm_client.generate(current_prompt,
            response_format=json_schema)
            raw_json = json.loads(raw_response)

            # 3. Pydantic-validointi ja automaattinen tyyppimuunnos (coercion)
            validated_data = DynamicModel(**raw_json)
            return validated_data.model_dump() # ONNISTUI! Puhdasta dataa.

        except (json.JSONDecodeError, ValidationError) as e:
            if attempt == max_retries - 1:
                # Fail-Fast, maksimiyritykset saavutettu (Circuit Breaker)
                raise RuntimeError(f"LLM epäonnistui noudattamaan skeemaa {max_retries} yrityksen jälkeen.
                Virhe: {e}")

            logging.warning(f"LLM Schema Error (Attempt {attempt+1}/{max_retries}): {e}")

    # 4. Self-Healing: Syötetään virhe takaisin LLM:lle
    error_msg = e.json() if isinstance(e, ValidationError) else str(e)
    correction_prompt = (
        f"\n\n[SYSTEM]: Your previous response contained invalid JSON. "
        f"Validation errors:\n{error_msg}\n"
        f"Please carefully correct the JSON output to strictly match the requested schema."
    )
    # Lisätään korjauspyyntö seuraavaan iteraatioon
    current_prompt += f"\n\n{raw_response}{correction_prompt}"

```

4. Työpaketit (Task Breakdown)

Tiketti	Kuvaus	Työmäärä
QUORUM-301	SchemaCompilerService ja Välimuistitus: Rakenna uusi palvelu (backend_v2/llm/schema_compiler.py), joka lukee työnkulun konfiguraatiosta (esim. matriisit) halutut ulostulokentät ja niiden tyypit. Toteuta tiukka lru_cache ja hashaustekniikka muistivuotojen estämiseksi. Kirjoita testi, joka generoi 10 000 mallia ja varmistaa assertioilla, ettei uusia luokkia luoda ohi välimuistin.	8 h
QUORUM-302	LLM API Structured Outputs: Päivitä LLM-asiakasohjelma (backend_v2/llm/provider.py tai client.py) injektoimaan Pydantic-mallin JSON Schema (.model_json_schema()) suoraan LLM-palveluntarjoajan API:in (esim. OpenAI:n response_format tai Tool Calling).	4 h
QUORUM-303	Self-Healing Retry Loop: Päivitä backend_v2/llm/handler.py toteuttamaan	8 h

	Retry-mekanismi (Circuit Breaker: esim. max_retries=3). Ota Pydanticin ValidationErrorit kiinni, formatoi ne korjauskehotteeksi ja yritä API-kutsua uudelleen.	
QUORUM-304	Hook-koodin siivous (Pilotti): Pilotoi ratkaisua numeerisissa ja rakenteellisissa koukuissa (esim. scoring.py ja metrics.py). Poista kaikki vanha, manuaalinen tyyppitarkistuskoodi (try/exceptit ja dict.get()), koska koukku voi nyt luottaa saavansa 100 % oikeanmuotoista dataa.	6 h
QUORUM-305	Integraatiotestit ja Observabiliteetti: Kirjoita testit, joissa mockattu LLM palauttaa ensin rikkinäistä JSONia. Varmista, että toistoluuppi aktivoituu, syöttää virheen takaisin, ja toisella (mockatulla) yrittämällä suoritus menee Pydanticista läpi. Lisää lokitus (Datadog/Sentry) Retry-tapahtumille LLM-mallien laadun seurantaa varten.	6 h

5. Hyväksymiskriteerit (Definition of Done)

- [] Abstraktit JSON-skeemat käännetään onnistuneesti vahvasti tyyplitetyiksi Pydantic-malleiksi lennosta.
- [] **Kriittinen:** Pydantic-mallien kääntämisessä on todistettavasti käytössä välimuisti (Caching), eikä uusia type-olioita luoda iteratiivisesti samasta skeemasta (estää muistivuodot / OOM).
- [] LLM API-kutsuihin injektoidaan JSON Schema -määrittely suoraan API-tasolla (Structured Outputs).
- [] Pydantic ValidationError ei aiheuta välitöntä 500-virhettä, vaan aktivoi LLM:n itsekorjausluupin (Self-Healing).
- [] Itsekorjausluupilla on ehdoton maksimiyritysten raja (Circuit Breaker). Sen ylittyminen aiheuttaa hallitun Fail-Fast -kaatumisen.
- [] Pilotoiduista koukuista (scoring.py, metrics.py) on poistettu manuaalinen tyyppien arvailu ja defensiivinen koodi.
- [] Yksikkö- ja integraatiotestit kattavat virheellisen LLM-vastauksen palautumisen, onnistuneen Retryn ja muistivuototurvallisuuden.

6. Riskit ja Mitigaatio

Riski	Vaikutus	Hallintakeino (Mitigation)
Muistivuodot (Memory Leak OOM): type-objektien hallitsematon lennosta luonti täyttää Worker-prosessin muistin.	Erittäin Kriittinen	Ratkaistu QUORUM-301 :ssä vaatimalla kryptografinen tiiviste skeemamäärittelystä ja pakottamalla välimuistin käyttö. Muistitestin läpäisy on koodikatselmoinnissa (Code Review) tärkein tarkistettava asia.
Kustannusräjähdykset (FinOps): LLM ei ymmärrä Pydanticin virhettä, vaan jatkaa väärän tiedon tuottamista, polttaen tuhansia API-tokeneita	Korkea	Ratkaistu QUORUM-303 :ssa kovakoodatulla max_retries -katolla. Lisäksi Pydanticin e.json() -tulosteesta karsitaan liiallinen metatieto ennen sen syöttämistä LLM:lle. Natiivi

sil mukassa.		Structured Outputs (QUORUM-302) vähentää alkuperäisten virheiden määrää radikaalisti, joten luuppiin joudutaan ylipäättään aniharvoin.
Skeemojen "Ylioptimointi" (Over-engineering): Yritämme validoida täysin vapaamuotoisia tekstivastauksia (esim. esseitä) aivan liian tiukalla Pydantic-skeemalla, jolloin validointi epäonnistuu jatkuvasti.	Keskiverto	Aloitetaan pilotilla (QUORUM-304). Tuetaan ensin primitiivityyppejä (str, float, int, bool) ja niiden listoja. Jätetään monimutkaisemmat rakenteet fallbackiksi tai tuetaan niitä dict[str, Any] -varaventtiilinä, kunnes moottorin käyttäytyminen on hioutunut tuotannossa.